Plan de pruebas de la aplicación Bicis

Historia de usuario: Detectar la ciudad automáticamente según mi ubicación (ID 532319).

Yo, como visitante temporal, quiero que la aplicación detecte la ciudad en la que estoy a partir de mi ubicación de manera que pueda ver sus estaciones nada más llegar, sin tener que buscar la ciudad en la lista.

Criterios de aceptación acordados con el Product Owner:

- **Prueba 1 (Éxito).** La ubicación del dispositivo está disponible. El usuario abre la aplicación. Se verifica que se muestra directamente la ciudad en la que se encuentra el usuario y sus estaciones.
- **Prueba 2 (Error).** No se puede obtener la ubicación del dispositivo. El usuario abre la aplicación. Se verifica que se muestra un mensaje indicando que no se ha podido cargar la ubicación y se muestran todas las ciudades para que el usuario elija una a mano.

Los niveles de prueba que se van a aplicar son los siguientes:

- Pruebas de aceptación. Se definirán a partir de los criterios de aceptación de la historia de usuario y se ejecutarán de forma manual en el Sprint Review.
- Pruebas de integración. La estrategia para la definición del orden de las pruebas de integración será jerárquica (ascendente). Se probará:
    - La integración entre MainPresenter y NetworkLocator con las clases de dominio (Network, Location). Se utilizará la técnica de prueba de métodos y caja negra, con JUnit y Mockito.
    - La integración entre las tres capas (prueba funcional completa): MainView, MainPresenter y JsonNetworksRepository, con la ubicación del dispositivo simulada. Se utilizará la técnica de casos de uso, con JUnit, Espresso y Hilt.
- Pruebas unitarias. Se aplicarán a la lógica nueva: la clase NetworkLocator y el presenter MainPresenter. Se utilizará la técnica de prueba de métodos, con técnicas de caja negra (partición equivalente y AVL), usando JUnit y Mockito.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel mencionado anteriormente.

PRUEBAS DE ACEPTACIÓN

En base a la historia de usuario se identifican los siguientes escenarios:

A1. HU: Detectar la ciudad automáticamente según mi ubicación
    a. Ubicación disponible: se abren directamente las estaciones de la ciudad (Prueba 1)
    b. Permiso de ubicación denegado (Prueba 2)
    c. Ubicación desactivada o no disponible (Prueba 2)

PRUEBAS UNITARIAS

Pruebas unitarias de la clase NetworkLocator

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba de cada método. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión.

Las redes de partida son las del Anexo 1 (con las coordenadas redondeadas a 4 decimales) y el radio máximo es de 20 km salvo que se indique otro valor.

- Método findNearestNetwork(networks, lat, lon, maxKm)

Tabla 1. Casos de prueba del método findNearestNetwork

Identificador	Entrada	Valor esperado
UNL.1a	(43.4600, -3.8000) (Usuario en Santander)	TUeBICI
UNL.1b	(40.4618, -3.7038) (A unos 5 km de Madrid)	BiciMAD
UNL.1c	(48.8566, 2.3522) (París, lejos de todas las redes)	null
UNL.1d	(40.4550, -3.7038) con otra red en (40.4500, -3.7038) (Dos redes en la misma ciudad)	La otra red (la más cercana)
UNL.1e	(43.4623, -3.8099) con redes sin location, con coordenadas null, fuera de rango o null (Datos incorrectos)	TUeBICI
UNL.1f	Lista vacía	null
UNL.1g	Lista null	null
UNL.1h	(43.3000, -3.8099), maxKm = distancia a TUeBICI y maxKm - 0,01 (Valor límite del radio)	TUeBICI / null

- Método distanceKm(lat1, lon1, lat2, lon2)

Identificador	Entrada	Valor esperado
UNL.2a	(43.4623, -3.8099), (40.4168, -3.7038) (Santander - Madrid)	338,8 km (± 1 km)
UNL.2b	(40.4168, -3.7038), (40.4168, -3.7038) (Mismo punto)	0 km

Pruebas unitarias del presenter MainPresenter

Se prueba el presenter aislado, con la vista y la ubicación simuladas con Mockito, comprobando las llamadas que hace a la vista. Los nombres de los métodos nuevos son una propuesta y se ajustarán a la implementación final.

- Método init y respuesta al permiso y a la ubicación

Identificador	Entrada	Valor esperado
UMP.1a	Permiso concedido, ubicación (43.4623, -3.8099)	showNetworkStations(TUeBICI)
UMP.1b	Permiso denegado	showLocationError()
UMP.1c	Permiso concedido, falla la ubicación	showLocationError()
UMP.1d	Falla la carga de redes	showLoadError() y no se pide el permiso

PRUEBAS DE INTEGRACIÓN

El orden de las pruebas y los casos de prueba a realizar serían los siguientes:

1. MainPresenter con NetworkLocator, Network y Location.
   Los casos de prueba son los mismos definidos como UMP.x en la sección de pruebas unitarias, aquí renombrados como IMP.x, pero usando la clase NetworkLocator real y las redes del Anexo 1.

2. MainView con MainPresenter, NetworkLocator y JsonNetworksRepository.
   Los casos de prueba se definen en base a los escenarios de las pruebas de aceptación y se automatizan con Espresso. Parten de los datos del Anexo 1, la ubicación del dispositivo se sustituye por una simulada con Hilt y el permiso se concede con GrantPermissionRule. Los casos de prueba son los de la Tabla 2.

Tabla 2. Casos de prueba de integración funcional completa

Identificador	Entrada	Resultado
A1.a	Ubicación simulada (43.4623, -3.8099) (Santander)	Se abre la lista de estaciones de TUeBICI
A1.c	La ubicación simulada devuelve un error	Mensaje "No se ha podido cargar tu ubicación" y lista con Bicing, BiciMAD y TUeBICI

ANEXO 1. DATOS DE PRUEBA

Los datos con los que se lanza la aplicación en modo debug (res/raw/networks.json), que es necesario conocer para las pruebas de integración y aceptación, son los que se muestran en la Tabla 3.

Tabla 3. Redes de bicicletas de prueba

Red	Ciudad	Latitud	Longitud
Bicing	Barcelona, ES	41.3850639	2.1734035
BiciMAD	Madrid, ES	40.4168	-3.7038
TUeBICI	Santander, ES	43.4623057	-3.8099803